UNIVERSITY OF THESSALY
SCHOOL OF ENGINEERING

DEPARTMENT OF ELECTRICAL
& COMPUTER ENGINEERING

Νευρο-Ασαφής Υπολογιστική (Neuro-Fuzzy Computing)
Fall Semester 2024-2025
Δημήτριος Κατσαρός (prof. Dimitrios Katsaros)

Problem-set 3rd : **Teamwork** (2-persons)

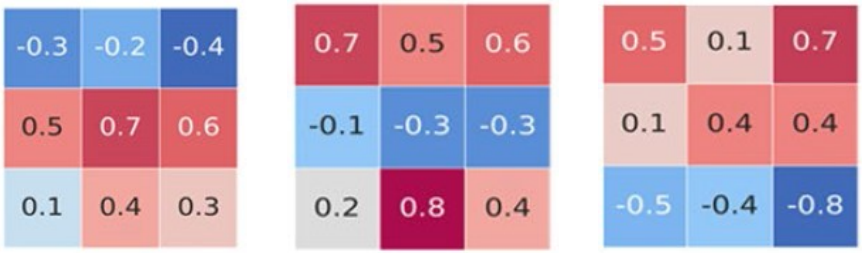
Announcement: Thursday, January 09, 2025
Delivery deadline: Sunday, February 09, 2025

SECTION 1: More on CNNs



Problem-01

You are given the following three kernels (filters) in a convolutional layer of a CNN.



Suppose you are asked to prune one of these filters in order to reduce the model size of the CNN, e.g., for purposes of running a TinyML application.

A. Which one would you prune based on the L_2 norm?
B. Which one would you prune based on some similarity measure among the three kernels? Define your similarity measure.

For both [A] and [B] show you analytic calculations.

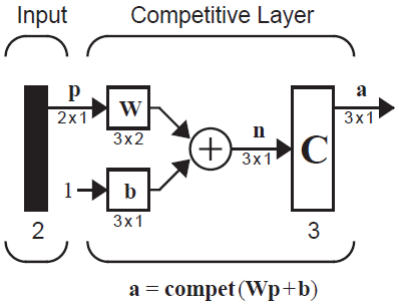
SECTION 2: More on Competitive learning



Problem-02

In the class lectures the term "**conscience**" was used to refer to a technique for avoiding the dead neuron problem plaguing competitive layers and LVQ networks.

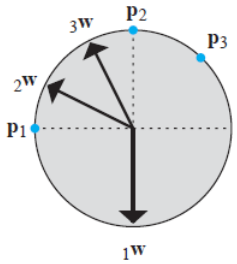
Neurons that are too far from input vectors to ever win the competition can be given a chance by using adaptive biases that get more negative each time a neuron wins the competition. The result is that neurons that win very often start to feel "guilty" until other neurons have a chance to win. The figure below shows a competitive network with biases.



A typical learning rule for the bias of neuron is:

$$b_i^{new} = \begin{cases} 0.9b_i^{old}, \text{ if } i \neq i^* \\ b_i^{old} - 0.2, \text{ if } i = i^* \end{cases}$$

A. Examine the vectors of the figure below.



Is there any order in which the vectors can be presented that will cause $_1$w to win the competition and move closer to one of the vectors? (Note: assume that adaptive biases are not being used.)

B. Given the input vectors and the initial weights and biases defined below, calculate the weights (using the Kohonen rule) and the biases (using the above bias rule). Repeat the sequence shown below until neuron 1 wins the competition

$$\mathbf{p}_1 = \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \mathbf{p}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \mathbf{p}_3 = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix}$$

$$_1\mathbf{w} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, {}_2\mathbf{w} = \begin{bmatrix} -2/\sqrt{5} \\ -1/\sqrt{5} \end{bmatrix}, {}_3\mathbf{w} = \begin{bmatrix} -1/\sqrt{5} \\ -2/\sqrt{5} \end{bmatrix}, b_1(0) = b_2(0) = b_3(0) = 0$$

Sequence of input vectors: p_1, p_2, p_3, p_1, p_2, p_3, ….,

C. How many presentations occur before $_1$w wins the competition?

SECTION 3: Recurrent neural networks



Problem-03

We define the following activation function NFC(x)= x tanh(e^x). Does this function present the vanishing and/or exploding gradients problem? Explain.



Problem-04

You are asked to generate an Auto Regressive (AR) model and then create an RNN (such as LSTM, or GRU) that predicts it. Generate samples of an Auto Regressive model of the form:

$$X_t = a_1 X_{t-1} + a_2 X_{t-2} + a_3 X_{t-3} + U_t$$

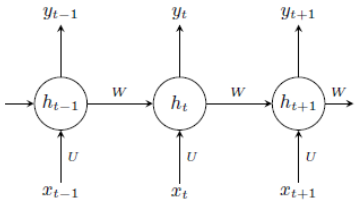
where a_1= 0.6, a_2= −0.5, a_3= -0.2 and U_t is independent-identically distributed Uniform in the interval (0, 0.1). Now train an RNN of your choice that predicts the sequence.

A. Apply the training algorithm on new samples and calculate the averaged cost square error cost function. Investigate different number of training samples. (You are not allowed to use your model knowledge when you design the RNN.)



Problem-05

Consider the following typical RNN:



Each state h_t is given by:

$$h_t = \sigma(W h_{t-1} + U x_t), \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

Let L be a loss function defined as the sum over the losses L_t at every time step until time T:

$$L = \sum_{t=0}^{T} L_t$$

where L_t is a scalar loss depending on h_t.

We want to derive the gradient of this loss function with respect to the parameter W.

A. Given $y=\sigma(Wx)$ where $y \in R^n$, $x \in R^d$ and $W \in R^{nxd}$. Derive the Jacobian $=diag(\sigma')W \in R^{nxd}$.

B. Derive the quantity:

$$\frac{\partial L}{\partial W} = \sum_{t=0}^{T} \sum_{k=1}^{t} \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W}$$

C. Write down $\partial L/\partial W$ as expanded sum for T = 3. You should see that if we want to backpropagate through n timesteps, we have to multiply the matrix *diag(σ')W* n times with itself.

D. Remember that any diagonalizable (square) matrix M can be represented by its eigendecomposition $M = Q\Lambda Q^{-1}$ where Q is a matrix whose i-th column corresponds to the i-th eigenvector of M and Λ is a diagonal matrix with the corresponding eigenvalues placed on the diagonals. Proof by induction that for such a matrix the product $\prod_{i=1}^{n} M$ can be written as: $M^n = Q\Lambda^n Q^{-1}$.

E. Consider the weight matrix $W = \begin{pmatrix} 0.58 & 0.24 \\ 0.24 & 0.72 \end{pmatrix}$. Its eigendecomposition is:

$$W = Q\Lambda\, Q^{-1} = \begin{pmatrix} 0.8 & -0.6 \\ 0.6 & 0.8 \end{pmatrix} \begin{pmatrix} 0.4 & 0 \\ 0 & 0.9 \end{pmatrix} \begin{pmatrix} 0.8 & 0.6 \\ -0.6 & 0.8 \end{pmatrix}$$

Calculate W^{30}. What do you observe? What happens in general if the absolute value of all eigenvalues of W is smaller than 1? What happens if the absolute value of any eigenvalue of W is larger than 1? What if all eigenvalues are 1?

SECTION 4: Fuzzy subsets theory



Problem-06

Consider the following reference set: {A, B, C, D, E, F, G}, and the fuzzy subsets

A= {(A|0), (B|0.3), (C|0.7), (D|1), (E|0), (F|0.2), (G|0.6)},

B= {(A|0.3), (B|1), (C|0.5), (D|0.8), (E|1), (F|0.5), (G|0.6)},

C= {(A|1), (B|0.5), (C|0.5), (D|0.2), (E|0), (F|0.2), (G|0.9)}.

Calculate the following:

A. $\mathbb{A} \cap \mathbb{B}$
B. $\mathbb{A} \cup \mathbb{B}$
C. $\mathbb{A} \cap \mathbb{B}^c$
D. $(\mathbb{A} \cup \mathbb{B}^c) \cap \mathbb{C}$
E. $(\mathbb{A} \cap \mathbb{B})^c \cup \mathbb{C}^c$
F. $(\mathbb{A} \cap \mathbb{A}^c) \cup \mathbb{A}$



Problem-07

Show that $N_\alpha(N_\alpha(x)) = x$ for the generalized negation operator

$$N_\alpha(x) = \frac{1-x}{1+\alpha x} \qquad \alpha>1,\ 0\le x<1.$$



Problem-08

Show that the following set does not satisfy the law of contradiction and the law of excluded middle:

$$\mu_A(x) = 1/(1+x).$$



Problem-09

Let t : **S**→[0,1] be a continuous or "fuzzy" truth function on the set **S** of statements. Define the implication operator as the truth function $t_L(A \to B)=\min(1, 1-t(A)+t(B))$ for statements A and B. Then prove the following generalized fuzzy *modus ponens* inference rule:

	$t_L(A \to B)$	=	c
	$t(A)$	≥	α
Therefore	$t(B)$	≥	max(0, α+c-1)

Hence, if $t(A)= t_L(A \to B)= 1$, then $t(B)=1$, which generalizes classical bivalent *modus ponens*.



Problem-10

Use the multivalued logic operations of the previous problem to prove the following generalized *modus tollens* inference rule:

	$t_L(A \to B)$	=	c
	$t(B)$	≤	b
Therefore	$t(A)$	≤	min(1, 1-c+b)

Hence, if $t_L(A \to B)= 1$ and $t(B)=0$, then $t(A)=0$, which generalizes classical bivalent *modus tollens*.



Problem-11

Lukasiewicz's multivalued or fuzzy logic ($\mathbf{L_1}$ logic) uses a continuous-valued truth function et t : **S**→[0,1] defined on the set **S** of statements. Lukasiewicz defined the generalized conjunction (AND), disjunction (OR), negation (NOT) operators respectively as:

$t(A \text{ AND } B)$	=	$\min(t(A), t(B))$
$t(A \text{ OR } B)$	=	$\max(t(A), t(B))$
$t(\text{NOT-}A)$	=	$1 - t(A)$

for statements *A* and *B*. Prove the generalized noncontradiction-excluded middle law:

$$t(A \text{ AND } \sim A) + t(A \text{ OR } \sim A) = 1.$$

Useful information:
The deadline is strict. It is possible to get an extension (up to 4 days), but you need to get the approval of the instructor, and this is going to cost a 10% penalty to the final grade of this Problem-Set. Delivery of the solutions' pdf (typeset or very very easy to read handwritten) is done by email to dkatsar@uth.gr. The subject of the message should be **strictly**: CE418-Problem set 03: AEM1-AEM2

Symbol interpretation:



It requires "algorithmic" and/or mathematical thinking.



It requires the development of code (in any language/platform you wish). The final deliverable should contain: a) The solution of the exercise, and b) The implementation source code.